



PARALLEL AND DISTRIBUTED COMPUTING

10 – esercitazione MPI (funzioni bloccanti)

Marco Lapegna
Dipartimento di Matematica e Applicazioni
Università degli Studi di Napoli Federico II

www.docenti.unina.it/marco.lapegna

1

Marco Lapegna
 Parallel and Distributed Computing
 10 – esercitazione MPI (funzioni bloccanti)

Problema

sia una matrice A di numeri reali di ordine $N=400$ inizializzata come in figura

Bordo (arancio)

prima riga: $A[0][j] = j$

ultima riga: $A[N-1][j] = N-j-1$

prima colonna: $A[i][0] = i$

ultima colonna: $A[i][N-1] = N-i-1$

Elementi centrali: (verde)

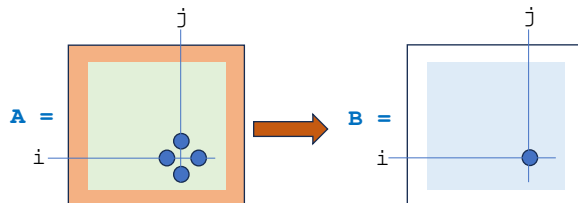
$A[i][j] = 0 \quad i = 1, \dots, N-2 \quad j = 1, \dots, N-2$

2

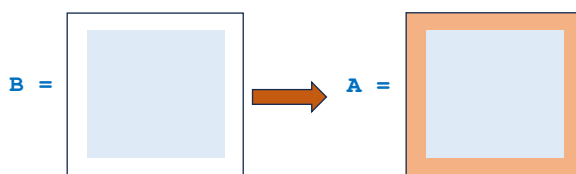
algoritmo sequenziale

- 1) calcolare gli **elementi interni** di una matrice **B** (cioè escluse prime e ultime righe e colonne) come segue

$$B[i][j] = (A[i-1][j] + A[i+1][j] + A[i-1][j] + A[i+1][j]) / * 0.25$$



- 2) ricopiare gli **elementi interni** di **B** su **quelli interni** di **A**



Iterare i passi 1) e 2) per 8000 volte

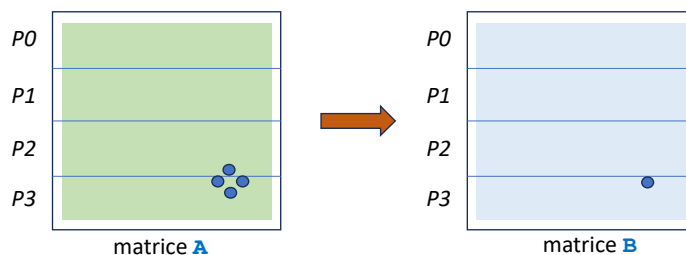
3

algoritmo parallelo a memoria distribuita

Detto **NP** il numero di processi, ogni processo possiede solo un blocco delle matrici **A** e **B**, di **N/NP** righe e **N** colonne

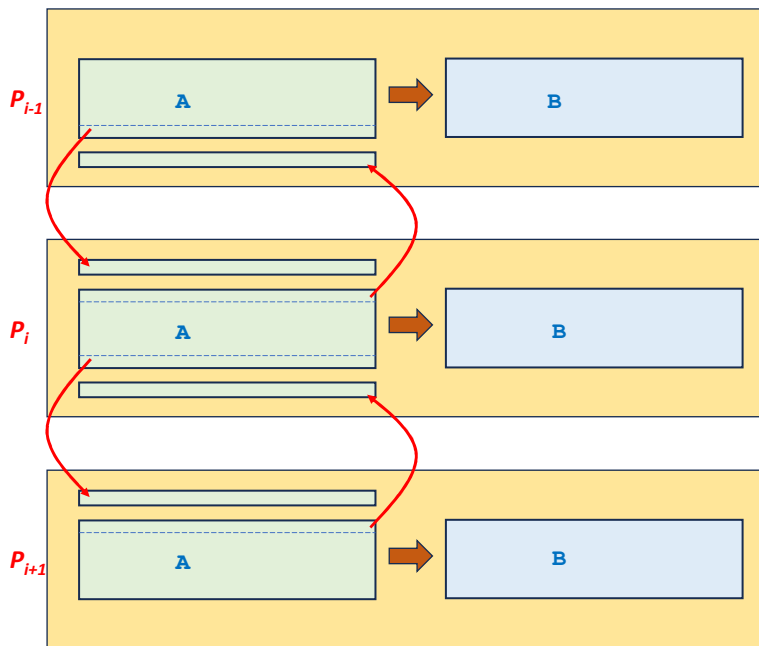
PROBLEMA

- ad ogni iterazione, ogni processo (tranne P0), per calcolare la prima riga della sua porzione di **B**, ha bisogno dell'ultima riga della porzione di **A** del processo precedente
- ad ogni iterazione, ogni processo (tranne P3), per calcolare l'ultima riga della sua porzione di **B**, ha bisogno della prima riga della porzione di **A** del processo successivo



4

lo scambio delle righe (dettaglio)



ad ogni iterazione, il processo P_i (tranne P_0), invia la prima riga della propria porzione di A al processo P_{i-1} , dal quale riceve l'ultima riga della sua porzione di A

ad ogni iterazione, il processo P_i (tranne P_{NP-1}), invia l'ultima riga della propria porzione di A al processo P_{i+1} , dal quale riceve la prima riga della sua porzione di A

5

algoritmo parallelo a memoria distribuita (versione SPMD)

- inizializza la propria porzione di A (attenzione ai processi con $id=0$ e $id=NP-1$ che possiedono prima e ultima riga)
- itera 8000 volte

- se $id \neq 0$ manda al processo $id-1$ la prima riga della propria porzione di A e ricevi la sua ultima riga
- se $id \neq NP-1$ manda al processo $id+1$ l'ultima riga della propria porzione di A e ricevi la sua prima riga

- calcola la parte interna della matrice B
- se $id \neq 0$ calcola anche la prima riga della propria porzione di B
- se $id \neq NP-1$ calcola anche l'ultima riga della propria porzione di B

- copia la parte interna della propria porzione di B su A
- se $id \neq 0$ copia anche la prima riga di B sulla prima riga di A
- se $id \neq NP-1$ copia anche l'ultima riga di B sull'ultima riga di A

- Tutti i processi eseguono lo stesso algoritmo
- Nessun processo possiede mai le matrici A e B per intero
- Implementare con MPI una versione bloccante (sono sufficiente le 6 funzioni base di MPI)

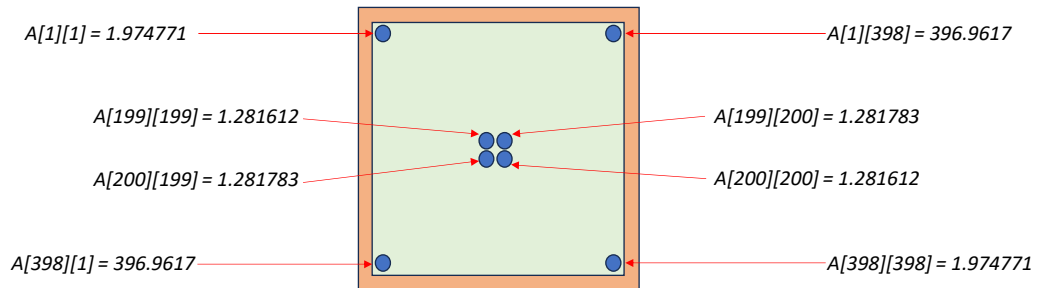
6

$N = 400$, $Niter = 8000$

NP=1			NP=2			NP=4			NP=8		
Tempo	Sp	Ep	Tempo	Sp	Ep	Tempo	Sp	Ep	Tempo	Sp	Ep
18.09	1	1	9.64	1.87	0.93	5.61	3.22	0.80	3.56	5.08	0.63

laplace

Un esempio di valori corretti

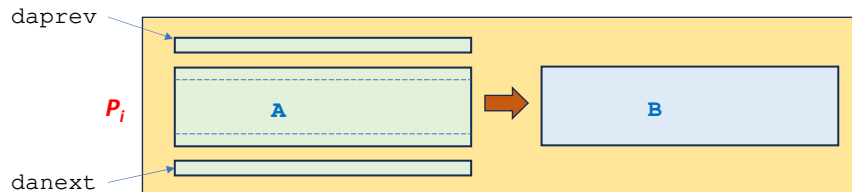


Alcuni valori della matrice A al termine del programma

7

```
void laplace(float *A, float *B, float *daprev, float *danext,
            int N, int LD, int Niter)
```

- A array contenente la matrice dei dati
- B array contenente la matrice di appoggio
- $daprev$, $danext$ array per lo scambio dei dati con i processi vicini
- N ordine della matrice
- LD leading dimensione degli array A e B
- $Niter$ numero di iterazioni



inserire solo la function in un file di nome `laplace.c`

8